
Active Directory Attack & Detection Lab

Technical Engineering Report

Detection Engineering Post-Mortem and Validation Report

Audience: SOC Leads, Detection Engineers, Technical Managers
Environment: AD-LAB / 10.10.10.0/24 / adlab.test
Core Platform: Windows Server 2022, Windows 11, Kali, Sysmon, Wazuh

Contents

1. Executive Summary & Scope	2
2. Environment & Telemetry Pipeline	2
2.4 Windows Advanced Audit Policy	5
2.5 PowerShell Script Block Logging	5
2.6 Sysmon	7
2.7 Wazuh Agent Collection	8
3. Attack & Detection Matrix	9
4. Deep-Dive Attack Scenarios & Detection Logic	10
4.1 Network Reconnaissance — Nmap	10
4.2 Password Spraying	13
4.3 Kerberoasting — svc_web	14
4.4 PowerShell Active Directory Reconnaissance	17
4.5 Privileged Group Escalation	20
4.6 Deception Mechanism — canary.admin	22
5. Supporting Validation — Remote Administrative Logon	23
6. Telemetry Gaps & Limitations	25
7. Detection Validation Summary	29
8. False-Positive Validation	30
9. Hardening & Post-Hardening Retest	30
10. Threat-Hunting Workflows	32
11. Engineering Recommendations	33
12. Source Artifacts Reviewed	34
13. Final Engineering Assessment	34
Security Disclaimer	35

Environment: AD-LAB — 10.10.10.0/24 — adlab.test

Report Type: Detection Engineering Post-Mortem and Validation Report

1. Executive Summary & Scope

The Active Directory Attack & Detection Lab is an isolated detection-engineering environment designed to validate the full defensive lifecycle around common identity, authentication, endpoint, and Active Directory attack patterns.

The lab uses Windows Server 2022, Windows 11, Kali Linux, Sysmon, PowerShell Script Block Logging, Wazuh SIEM, custom Wazuh XML rules, Sigma detection-as-code, Windows Advanced Audit Policy, and targeted Directory Service auditing.

The engineering workflow is:

Generate Activity

↓

Generate Endpoint Telemetry

↓

Collect / Ingest

↓

Decode / Correlate

↓

Alert / Hunt

↓

Validate

↓

Tune

↓

Harden

↓

Retest

The core environment is:

Domain:	adlab.test
NetBIOS:	ADLAB
Network:	AD-LAB
Subnet:	10.10.10.0/24
DC:	DC01

The project successfully validated detections for network reconnaissance, password spraying, Kerberoasting, suspicious PowerShell activity, privileged-group changes, and authentication against a canary identity. It also validated Active Directory object-access telemetry through Event ID 4662.

Two primary telemetry gaps remain documented:

1. A controlled remote administrative logon generated Windows Event ID 4624, Logon Type 3, but an exact dedicated Wazuh alert was not confirmed.
2. LDAP diagnostic Event ID 1644 was generated locally on DC01, but Wazuh ingestion was not confirmed.

The repository also contains a PowerShell detection-content mismatch that should be corrected: the current Wazuh rule 110120 detects suspicious script-block execution patterns, while the Sigma rule powershell-ad-recon.yml detects Get-AD* Active Directory discovery commands.

2. Environment & Telemetry Pipeline

2.1 Host Inventory

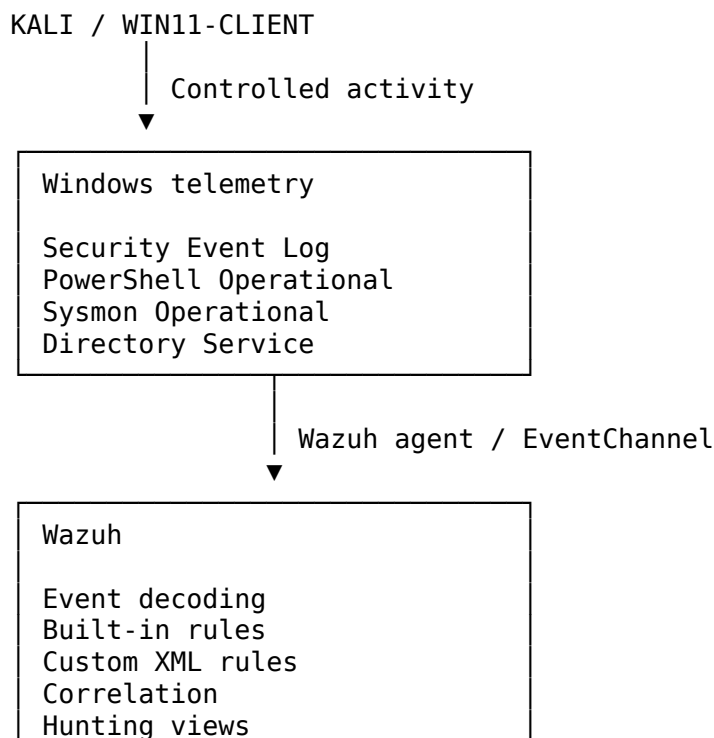
Host	OS / Platform	Role	IP Address	Security Function
DC01	Windows Server 2022	Domain Controller, DNS	10.10.10.10	Authentication, Kerberos, AD object auditing, privileged-group telemetry
WIN11-CLIENT	Windows 11	Domain workstation	10.10.10.20	User activity, PowerShell telemetry, Sysmon telemetry, remote administration source
WAZUH	Wazuh virtual appliance / manager platform	SIEM, Manager, Indexer, Dashboard	10.10.10.30	Event ingestion, correlation, custom rules, hunting
KALI	Kali Linux	Controlled testing workstation	10.10.10.50	Network reconnaissance and controlled attack simulation

The current repository identifies Wazuh as a Wazuh appliance / manager platform; it does not explicitly document the underlying appliance OS in the source artifacts used for this report.

2.2 Architecture

Figure 1 — High-level AD-LAB architecture showing the isolated Active Directory environment and the Wazuh monitoring path.

2.3 Telemetry Flow



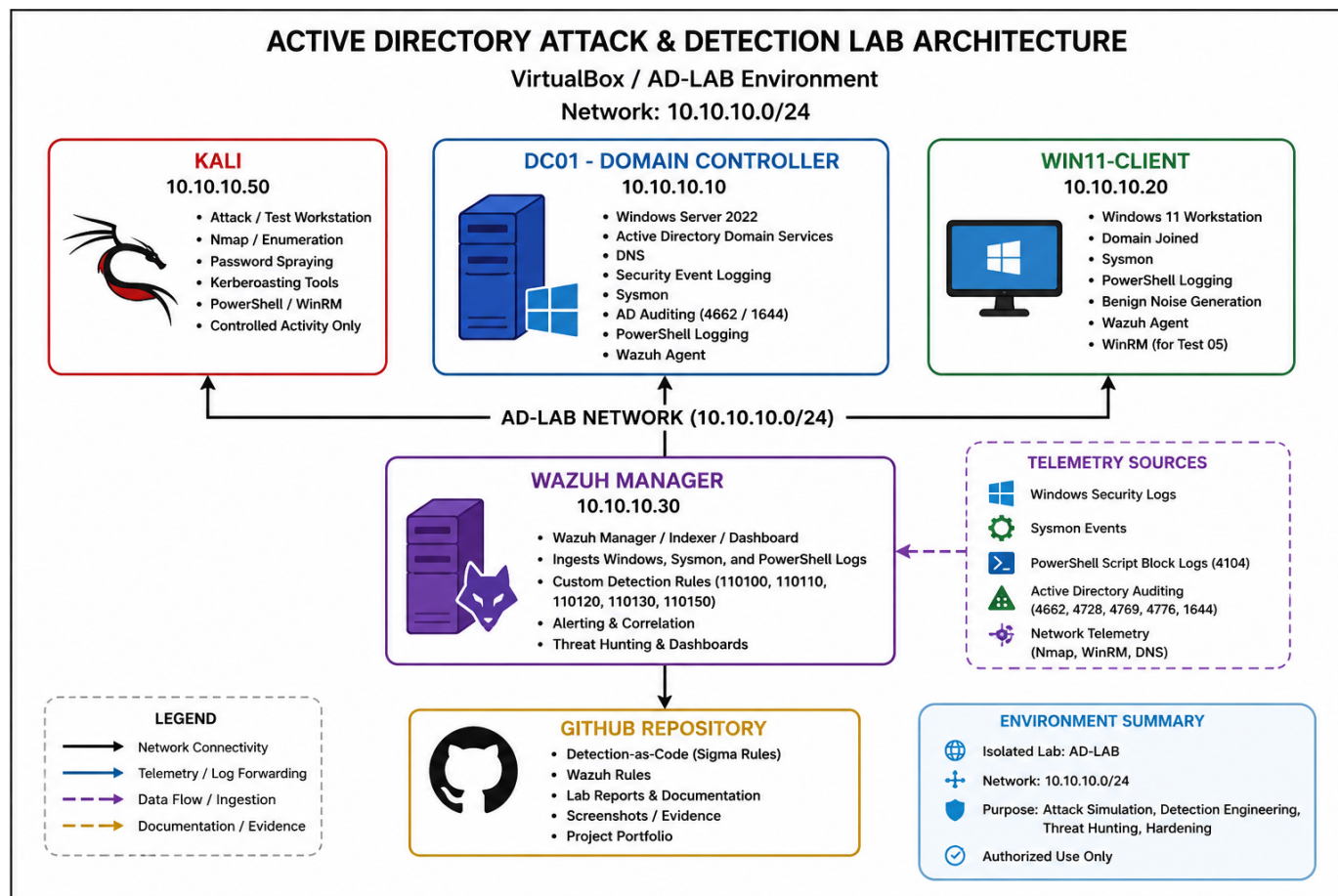
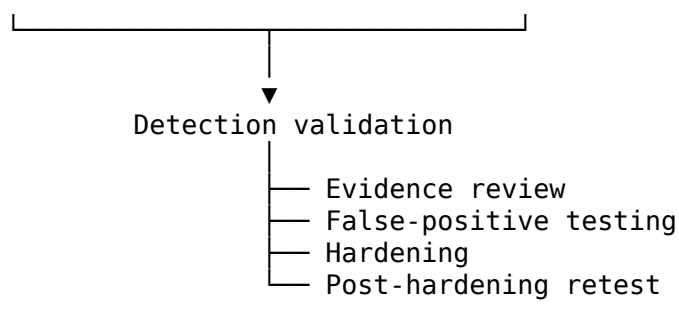


Figure 1: AD-LAB Architecture



2.4 Windows Advanced Audit Policy

The repository documents Advanced Audit Policy configuration for both DC01 and WIN11-CLIENT.

Key audit areas include:

Audit Area	Configuration / Purpose	Representative Events
Credential Validation	Success and Failure	4776
Kerberos Authentication Service	Success and Failure	4768, 4771
Kerberos Service Ticket Operations	Success and Failure	4769
Logon	Success and Failure	4624, 4625
Special Logon	Success	4672
Account Management	Security-group and account changes	4728, 4732, 4756
Process Creation	Success	4688
Directory Service Access	Success and Failure	4662
Directory Service Changes	Success	5136

Configuration reference:

configs/windows-auditing/advanced-audit-policy.md

Audit Evidence

Figure 3 — Advanced Audit Policy evidence. The important engineering point is that authentication, Kerberos, account-management, process, and Directory Service categories were enabled to create the event sources required by the detection scenarios.

2.5 PowerShell Script Block Logging

PowerShell Script Block Logging was enabled on the Windows systems through Group Policy.

Event channel:

Microsoft-Windows-PowerShell/Operational

Primary event:

4104

Representative controlled discovery commands documented in the repository include:

```

Get-ADDomain
Get-ADUser -Filter *
Get-ADGroup -Filter *
Get-ADGroupMember "Domain Admins"

```

```

PS C:\Users\Administrator> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

PS C:\Users\Administrator> auditpol /get /category:*
System audit policy
Category/Subcategory          Setting
System
  Security System Extension    No Auditing
  System Integrity             No Auditing
  IPsec Driver                 No Auditing
  Other System Events          No Auditing
  Security State Change        No Auditing
Logon/Logoff
  Logon                        Success and Failure
  Logoff                       No Auditing
  Account Lockout              No Auditing
  IPsec Main Mode              No Auditing
  IPsec Quick Mode             No Auditing
  IPsec Extended Mode          No Auditing
  Special Logon                 Success
  Other Logon/Logoff Events     No Auditing
  Network Policy Server         No Auditing
  User / Device Claims          No Auditing
  Group Membership              No Auditing
Object Access
  File System                  No Auditing
  Registry                     No Auditing
  Kernel Object                No Auditing
  SAM                          No Auditing
  Certification Services       No Auditing
  Application Generated         No Auditing
  Handle Manipulation           No Auditing
  File Share                    No Auditing
  Filtering Platform Packet Drop No Auditing
  Filtering Platform Connection No Auditing
  Other Object Access Events    No Auditing
  Detailed File Share           No Auditing
  Removable Storage             No Auditing
  Central Policy Staging        No Auditing
Privilege Use
  Non Sensitive Privilege Use   No Auditing
  Other Privilege Use Events     No Auditing
  Sensitive Privilege Use       No Auditing
Detailed Tracking
  Process Creation              Success

```

Figure 2: Advanced Audit Policy

Configuration reference:

configs/windows-auditing/powershell-logging.md

PowerShell Evidence

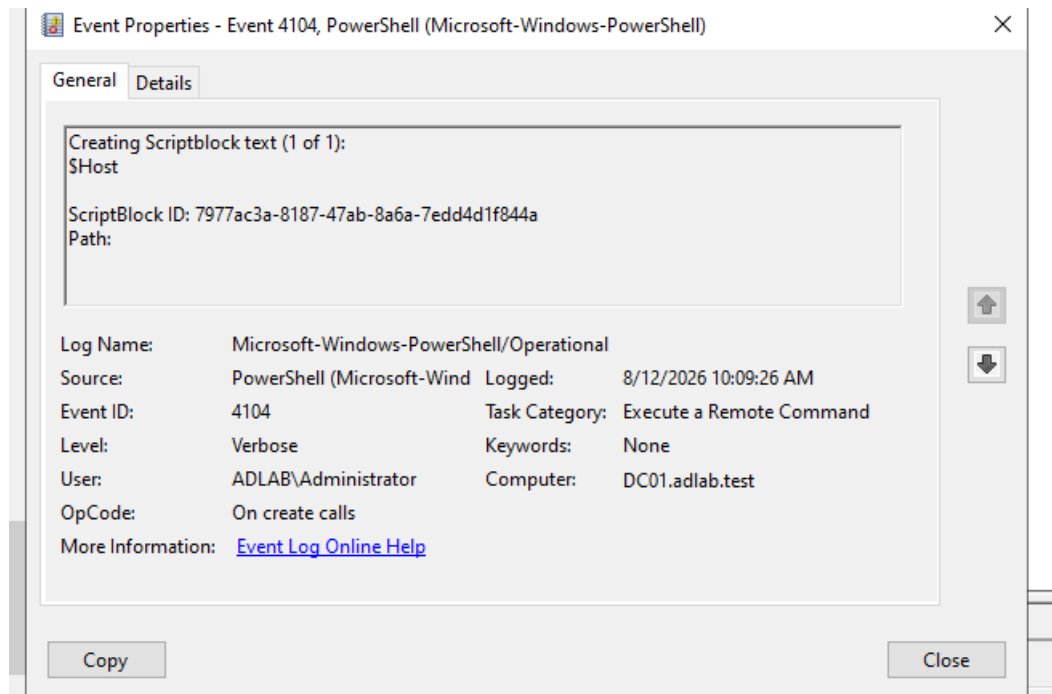


Figure 3: PowerShell 4104

Figure 4 — PowerShell Event ID 4104 evidence. The field of interest is the script-block content, which enables analysts to evaluate what PowerShell actually executed rather than alerting on powershell.exe alone.

2.6 Sysmon

The repository Sysmon configuration uses schema version 4.90 and configures SHA-256 and Import Hash calculation.

The configuration is intentionally broad and provides visibility for:

Sysmon Event	Purpose
1	Process creation
3	Network connection
11	File creation
22	DNS query

Configuration reference:

configs/sysmon/sysmon-config.xml

Condensed configuration excerpt:

```
<HashAlgorithms>sha256,imphash</HashAlgorithms>
```

```
<EventFiltering>
```

```
  <ProcessCreate onmatch="exclude" />
```



```

<NetworkConnect onmatch="exclude" />
<FileCreate onmatch="exclude" />
<DnsQuery onmatch="exclude" />
</EventFiltering>

```

Sysmon Evidence

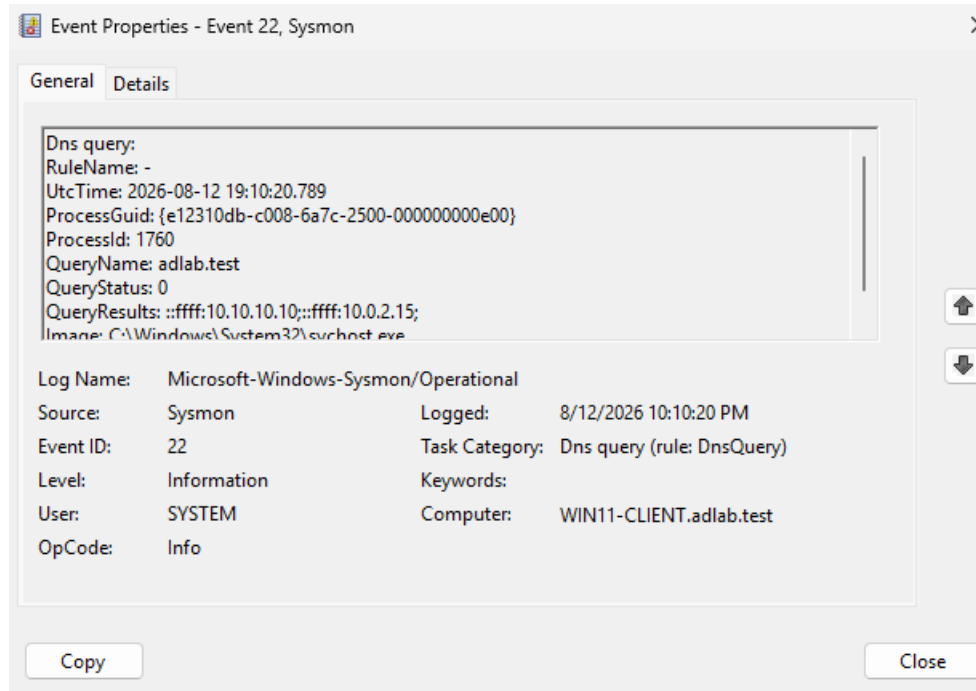


Figure 4: Sysmon DNS Event

Figure 5 — Sysmon Event ID 22 on WIN11-CLIENT, demonstrating DNS-query telemetry available for endpoint hunting and baseline analysis.

2.7 Wazuh Agent Collection

The repository `agent.conf` explicitly configures EventChannel collection for:

Microsoft-Windows-PowerShell/Operational

Microsoft-Windows-Sysmon/Operational

Condensed configuration:

```

<agent_config os="Windows">
  <localfile>
    <location>Microsoft-Windows-PowerShell/Operational</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
    <location>Microsoft-Windows-Sysmon/Operational</location>
    <log_format>eventchannel</log_format>
  </localfile>
</agent_config>

```

Windows Security telemetry is clearly present in the project evidence and validation reports. However, Security-log collection is not explicitly defined in the current repository copy of `con-`

figs/wazuh/agent.conf. This is a reproducibility/documentation gap and should be corrected or explicitly documented as relying on the endpoint's existing/default Wazuh configuration.

Wazuh Agent Evidence

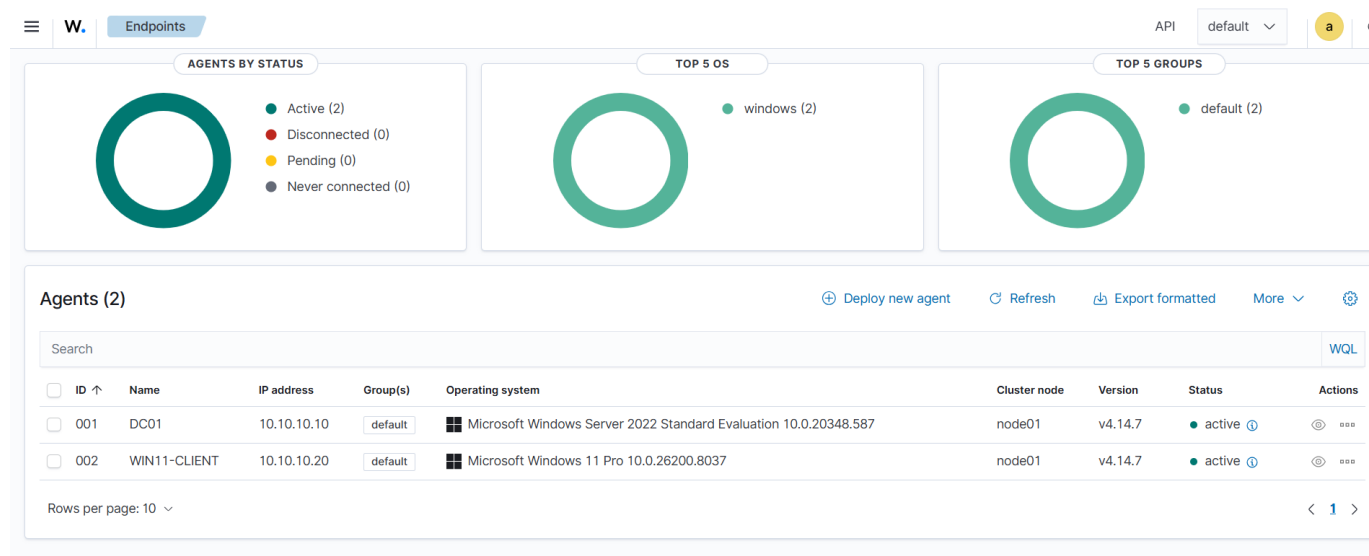


Figure 5: Wazuh Agents

Figure 6 — Wazuh agent status showing the monitored Windows systems connected to the SIEM.

3. Attack & Detection Matrix

Scenario	MITRE ATT&CK	Target Host / Identity	Generated Telemetry	Detection	Severity	Visual Reference
Network Reconnaissance	T1046 — Network Service Discovery	DC01 / no user identity required	Sysmon Event 3 network connections	Custom Wazuh 110129 helper + 110130 correlation	110129: Level 1 / no-log; 110130: Level 10	day4-wazuh-network-scan-tuned-detection.png
Password Spraying	T1110.003 — Password Spraying	Multiple AD lab identities	4625, 4771, 4776 depending on authentication path	Custom Wazuh 110110	Level 12	day3-wazuh-password-spray-dashboard-alert.png
Kerberoasting	T1558.003 — Kerberoasting	DC01 / svc_web	4769, RC4 0x17 context	Custom Wazuh 110100	Level 10	day3-kerberoasting-event4769-rc4.png, day3-wazuh-kerberoasting-live-detection.png

Scenario	MITRE ATT&CK	Target Host / Identity	Generated Telemetry	Detection	Severity	Visual Reference
PowerShell AD Recon	T1087.002 — Domain Account Discovery; T1069.002 — Domain Groups	adlab.test directory / lab Windows host	PowerShell 4104, Sysmon 1	Sigma AD-recon rule; related Wazuh 110120 is not equivalent	Sigma: Medium; Wazuh 110120: Level 10	day3-powershell-4104-event.png, day3-wazuh-hunting-powershell.png
Privileged Group Escalation	T1098 — Account Manipulation	DC01; helpdesk.test → Domain Admins	4728	Wazuh built-in 60159; Sigma privileged-group rule exists	Wazuh Level 12; Sigma High	test-06-group-change-4728.png, test-06-domain-admins-wazuh-rule-60159.png
Canary / Deception	T1078 — Valid Accounts context	canary.admin, source WIN11-CLIENT	4776	Custom Wazuh 110150	Level 12	advanced-canary-account-4776.png, advanced-canary-custom-rule-110150.png
Remote Administrative Logon	No ATT&CK mapping asserted in repository	WIN11-CLIENT → DC01, Administrator	4624, Logon Type 3	No exact dedicated alert confirmed	N/A	test-05-remote-logon-4624.png

4. Deep-Dive Attack Scenarios & Detection Logic

4.1 Network Reconnaissance — Nmap

Objective

Detect internal service enumeration from Kali against the Domain Controller.

Source:

KALI
10.10.10.50

Target:

DC01
10.10.10.10

Representative controlled command:

```
nmap -sV -Pn 10.10.10.10
```

Telemetry

The detection pipeline uses inbound Sysmon network-connection telemetry.

The current XML uses a two-stage design:

1. 110129 identifies inbound Sysmon Event 3 candidates and suppresses those helper events from normal alert output.
2. 110130 correlates repeated connections from the same source to the same destination across different destination ports.

Detection Logic

Condensed representation of the repository logic:

```
<rule id="110129" level="1">
  <field name="win.system.eventID">^3$</field>
  <field name="win.eventdata.initiated">^false$</field>
  <options>no_log</options>
</rule>

<rule id="110130" level="10" frequency="5" timeframe="10">
  <if_matched_sid>110129</if_matched_sid>
  <same_field>win.eventdata.sourceIp</same_field>
  <same_field>win.eventdata.destinationIp</same_field>
  <different_field>win.eventdata.destinationPort</different_field>
</rule>
```

Engineering Interpretation

This is stronger than alerting on a single connection because the logic requires a repeated multi-port pattern.

The meaningful correlation dimensions are:

Same source IP
Same destination IP
Different destination ports
5 matches
10-second window

Evidence

Figure 7 — Controlled Nmap enumeration of services exposed by DC01. The image establishes the source-side activity used to generate the detection test.

Figure 8 — Wazuh evidence for the tuned network-service discovery rule. The important threshold is the correlation of multiple inbound connections across different destination ports rather than a single network event.

Validation Result

Detected
Custom Rule: 110130
Level: 10
MITRE: T1046

The repository's benign-noise report further records that normal workstation traffic did not produce an observed 110130 false positive during the controlled validation window.

```
—(kali@kali)-[~]
$ nmap -Pn -sT -p 53,88,135,139,389,445,464,636,3268,3269 10.10.10.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-14 17:32 EDT
Nmap scan report for 10.10.10.10
Host is up (0.0017s latency).

PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
636/tcp   open  ldapssl
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl

Nmap done: 1 IP address (1 host up) scanned in 13.04 seconds
```

Figure 6: Kali Service Reconnaissance

```
{
  "/var/ossec/logs/alerts/alerts.json"
{
  "timestamp": "2026-08-15T14:47:37.652+0000",
  "ruleId": "110130",
  "level": 10,
  "description": "ADLAB: Possible network service scanning from 10.10.10.50 against 10.10.10.10",
  "sourceIp": "10.10.10.50",
  "destinationIp": "10.10.10.10",
  "destinationPort": "3268"
}
```

Figure 7: Wazuh Tuned Network Detection

4.2 Password Spraying

Objective

Identify repeated failed authentication activity from the same source against different domain users.

Expected / Observed Telemetry

Relevant Windows events documented in the project:

4625 – Failed logon

4771 – Kerberos pre-authentication failure

4776 – Credential validation

The exact event depends on the authentication path.

Detection Logic

The current Wazuh rule uses correlation:

```
<rule id="110110" level="12" frequency="4" timeframe="120">
  <same_field>win.eventdata.ipAddress</same_field>
  <different_field>win.eventdata.targetUserName</different_field>
</rule>
```

The design intent is:

4 related failures

within 120 seconds

same source IP

different target usernames

MITRE mapping:

T1110.003 – Password Spraying

Evidence

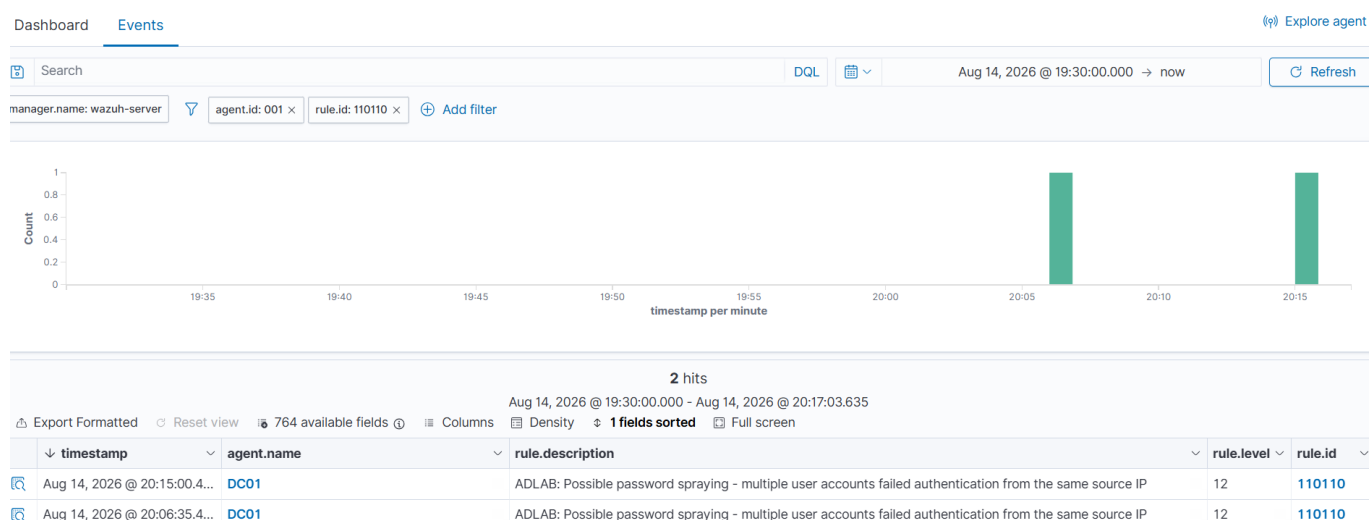


Figure 8: Password Spray Wazuh Alert

Figure 9 — Wazuh dashboard evidence for the password-spray correlation rule. The engineering value is in the rule metadata and correlation result rather than treating an isolated password failure as an attack.

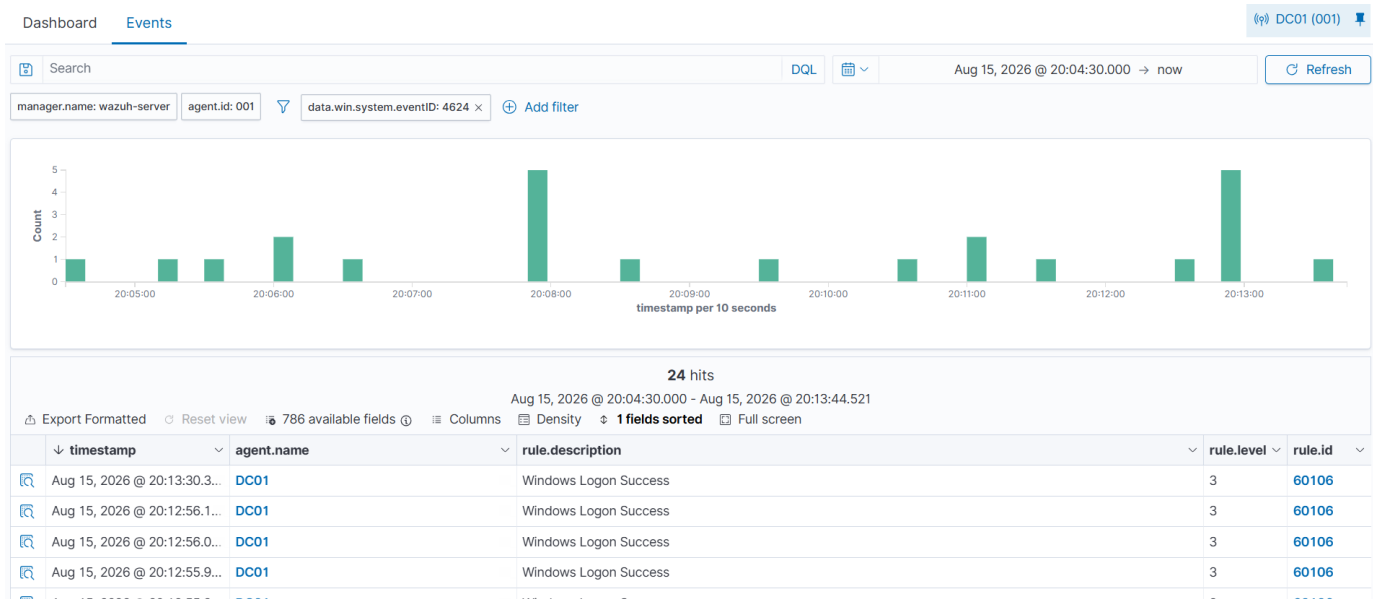


Figure 9: Authentication Hunting View

Figure 10 — Authentication hunting view used to pivot across 4624, 4625, 4771, and 4776 and inspect user, source, workstation, status, rule ID, and severity.

Validation Result

Detected
 Custom Rule: 110110
 Level: 12
 MITRE: T1110.003

Tuning Considerations

Potential legitimate causes of repeated failures include:

- Expired passwords
- Cached credentials
- Mapped drives
- Scheduled tasks
- Misconfigured services
- User typing errors

Recommended tuning dimensions:

Source IP
 Target-user count
 Failure frequency
 Authentication protocol
 Known service accounts
 Approved management systems

4.3 Kerberoasting — svc_web

Objective

Detect an RC4 Kerberos service-ticket request associated with the lab service account.

Service identity:

svc_web

SPN:

HTTP/web.adlab.test

Primary Telemetry

4769 – A Kerberos service ticket was requested

The custom Wazuh logic uses:

Event ID = 4769

Ticket encryption type = 0x17

Non-machine target identity

Detection Logic

Condensed XML:

```
<rule id="110100" level="10">
  <field name="win.system.eventID">^4769$</field>
  <field name="win.eventdata.ticketEncryptionType">^0x17$</field>
  <field name="win.eventdata.targetUserName" type="pcre2">^[^$]+$</field>
</rule>
```

MITRE mapping:

T1558.003 – Kerberoasting

Sigma Representation

The repository also contains:

detections/sigma/kerberoasting.yml

Its logic represents the same general pattern:

```
selection:
  EventID: 4769
  TicketEncryptionType: '0x17'
```

```
condition: selection and not filter_machine_accounts
```

Evidence

Figure 11 — Windows Kerberos service-ticket evidence. The key fields are Event ID 4769, the service-account context, and RC4 encryption type 0x17.

Figure 12 — Live Wazuh detection evidence for custom rule 110100, validating that the relevant Kerberos event reached the SIEM and matched the rule.

Validation Result

Detected

Custom Rule: 110100

Level: 10

MITRE: T1558.003

Engineering Note

Event 4769 is common in healthy Active Directory environments.


```

TimeCreated : 8/14/2026 9:34:38 AM
Message      : A Kerberos service ticket was requested.

Account Information:
  Account Name:      Administrator@ADLAB.TEST
  Account Domain:    ADLAB.TEST
  Logon GUID:        {4a4fa93a-f9df-71a0-d8e1-ae2ca0f2b93d}

Service Information:
  Service Name:      DC01$
  Service ID:        S-1-5-21-36052701-3025511810-1330942903-1000

Network Information:
  Client Address:    ::1
  Client Port:       0

Additional Information:
  Ticket Options:    0x40810000
  Ticket Encryption Type: 0x12
  Failure Code:      0x0
  Transited Services: -

This event is generated every time access is requested to a resource such as a computer or a Windows
service. The service name indicates the resource to which access was requested.

This event can be correlated with Windows logon events by comparing the Logon GUID fields in each event.
The logon event occurs on the machine that was accessed, which is often a different machine than the
domain controller which issued the service ticket.

Ticket options, encryption types, and failure codes are defined in RFC 4120.

```

PS C:\Users\Administrator> _

Figure 10: Kerberoasting 4769 RC4

```

[wazuh-user@wazuh-server ~]$ sudo grep "id":"110100" /var/ossec/logs/alerts/alerts.json | tail -n 5
{"timestamp":"2026-08-14T16:59:45.534+0000","rule":{"level":10,"description":"ADLAB: Possible Kerberoasting - RC4 Kerberos service ticket requested by non-m
achine account Administrator@ADLAB.TEST","id":"110100","mitre":{"id":["T1558.003"],"tactic":["Credential Access"],"technique":["Kerberoasting"]},"firedtimes
":1,"mail":false,"groups":["adlab","windows","kerberos","kerberoasting","credential_access"],"agent":{"id":"001","name":"DC01","ip":"10.10.10.10"},"manager
":{"name":"wazuh-server"},"id":"1786726785.4124213","full_log":{"win":{"system":{"providerName":"Microsoft-Windows-Security-Auditing"},"providerGuid
d":"{54849625-5478-4994-a5ba-3e3b0328c30d}"},"eventID":"4769"},"version":"0","level":"0","task":"14337","opcode":"0","keywords":"0x8
020000000000000","systemTime":"2026-08-14T16:59:15.6188453Z"},"eventRecordID":"11228","processID":"628","threadID":"2052","channel":"Security","computer":"DC01.adlab.test"},"severityValue":"AUDIT_SUCCESS"},"message":"A Kerberos service ticket was requested.\r\n\r\nAccount In
formation:\r\n\r\n\tAccount Name:\t\t\tAdministrator@ADLAB.TEST\r\n\r\n\tAccount Domain:\t\t\tADLAB.TEST\r\n\r\n\tLogon GUID:\t\t\t{f711d186-abfb-074e-5a80-e10
f371dde10}\r\n\r\n\tService Information:\r\n\r\n\tService Name:\t\t\ttsvc_web\r\n\r\n\tService ID:\t\t\tS-1-5-21-36052701-3025511810-1330942903-1107\r\n\r\n\t
\r\n\r\n\tNetwork Information:\r\n\r\n\tClient Address:\t\t\t::ffff:10.10.10.20\r\n\r\n\tClient Port:\t\t\t55752\r\n\r\n\tAdditional Information:\r\n\r\n\tTicket Op
tions:\t\t\t0x40810000\r\n\r\n\tTicket Encryption Type:\t\t0x17\r\n\r\n\tFailure Code:\t\t\t0x0\r\n\r\n\tTransited Services:\t\t-\r\n\r\n\r\nThis event is genera
ted every time access is requested to a resource such as a computer or a Windows service. The service name indicates the resource to which access was requ
sted.\r\n\r\n\r\nThis event can be correlated with Windows logon events by comparing the Logon GUID fields in each event. The logon event occurs on the mac
hine that was accessed, which is often a different machine than the domain controller which issued the service ticket.\r\n\r\n\r\nTicket options, encryption
types, and failure codes are defined in RFC 4120.\r\n\r\n"},"eventdata":{"targetUserName":"Administrator@ADLAB.TEST","targetDomainName":"ADLAB.TEST",
"serviceName":"svc_web","serviceSid":"S-1-5-21-36052701-3025511810-1330942903-1107","ticketOptions":"0x40810000","ticketEncryptionType":"0x1
7","ipAddress":"::ffff:10.10.10.20","ipPort":"55752","status":"0x0","logonGuid":"{f711d186-abfb-074e-5a80-e10f371dde10}"},"decoder":{"na
me":"windows_eventchannel"},"data":{"win":{"system":{"providerName":"Microsoft-Windows-Security-Auditing"},"providerGuid":"{54849625-5478-4994-a5ba-3e3b0328c
30d"},"eventID":"4769","version":"0","level":"0","task":"14337","opcode":"0","keywords":"0x8020000000000000","systemTime":"2026-08-14T16:59:15.6188453Z"},"ev
entRecordID":"11228","processID":"628","threadID":"2052","channel":"Security","computer":"DC01.adlab.test","severityValue":"AUDIT_SUCCESS"},"message":"A Ke
rberos service ticket was requested.\r\n\r\nAccount Information:\r\n\r\n\tAccount Name:\t\t\tAdministrator@ADLAB.TEST\r\n\r\n\tAccount Domain:\t\t\tADLAB.TEST\r\n\r\n\tLo
gon GUID:\t\t\t{f711d186-abfb-074e-5a80-e10f371dde10}\r\n\r\n\tService Information:\r\n\r\n\tService Name:\t\t\ttsvc_web\r\n\r\n\tService ID:\t\t\tS-1-5-21-36052701-3025511810
-1330942903-1107\r\n\r\n\tNetwork Information:\r\n\r\n\tClient Address:\t\t\t::ffff:10.10.10.20\r\n\r\n\tClient Port:\t\t\t55752\r\n\r\n\tAdditional Information:\r\n\r\n\tTicke
t Options:\t\t\t0x40810000\r\n\r\n\tTicket Encryption Type:\t\t0x17\r\n\r\n\tFailure Code:\t\t\t0x0\r\n\r\n\tTransited Services:\t\t-\r\n\r\n\r\nThis event is genera
ted every time access is requested to a resource such as a computer or a Windows service. The service name indicates the resource to which access was requested.\r\n\r\n\r\nTh
is event can be correlated with Windows logon events by comparing the Logon GUID fields in each event. The logon event occurs on the machine that was acces
sed, which is often a different machine than the domain controller which issued the service ticket.\r\n\r\n\r\nTicket options, encryption types, and failure cod
es are defined in RFC 4120.\r\n\r\n"},"eventdata":{"targetUserName":"Administrator@ADLAB.TEST","targetDomainName":"ADLAB.TEST","serviceName":"svc_web","serviceSid
":"S-1-5-21-36052701-3025511810-1330942903-1107","ticketOptions":"0x40810000","ticketEncryptionType":"0x17","ipAddress":"::ffff:10.10.10.20","ipPort":"55752
","status":"0x0","logonGuid":"{f711d186-abfb-074e-5a80-e10f371dde10}"},"location":"EventChannel"}
[wazuh-user@wazuh-server ~]$

```

Figure 11: Kerberoasting Live Detection

A production-quality rule should therefore use context such as:

- Service account
- SPN
- Encryption type
- Requesting identity
- Source address
- Request volume
- Known legacy services

After testing, the intentionally weak lab password for svc_web was replaced with a stronger unique password while the account remained non-administrative.

4.4 PowerShell Active Directory Reconnaissance

Objective

Generate and investigate Active Directory discovery activity through PowerShell.

Documented commands include:

```
Get-ADDomain
Get-ADUser -Filter *
Get-ADGroup -Filter *
Get-ADGroupMember "Domain Admins"
```

Telemetry

PowerShell Event 4104
Sysmon Event 1
Windows Event 4688 where configured

Sigma Detection

The current Sigma rule specifically covers Active Directory discovery commands:

```
selection:
  ScriptBlockText|contains:
    - 'Get-ADDomain'
    - 'Get-ADUser'
    - 'Get-ADGroup'
    - 'Get-ADGroupMember'
```

condition: selection

MITRE mappings:

T1087.002 – Domain Account Discovery
T1069.002 – Domain Groups

Sigma severity:

medium

Current Wazuh Rule Mismatch

The current operational Wazuh rule 110120 is related to PowerShell but is **not equivalent** to the Sigma AD-recon rule.

Its current matching logic focuses on suspicious execution/download/encoding terms:

```
<rule id="110120" level="10">
  <field name="win.eventdata.scriptBlockText"
        type="pcre2">
    (?i)(Invoke-Expression|\bIEX\b|DownloadString|Net\.WebClient|FromBase64String)
  </field>
</rule>
```

MITRE mapping for the current Wazuh rule:

T1059.001 – PowerShell

Engineering Assessment

The repository reports describe the AD-recon scenario as detected with rule 110120, while the current XML implementation detects a different set of suspicious PowerShell behaviors.

This should be resolved in one of two ways:

1. Modify 110120 to explicitly cover the Get-AD* discovery logic documented in the reports; or
2. Preserve 110120 as suspicious PowerShell execution and introduce a separate rule ID for AD discovery.

The second approach is preferable because it preserves two distinct analytical behaviors.

Evidence

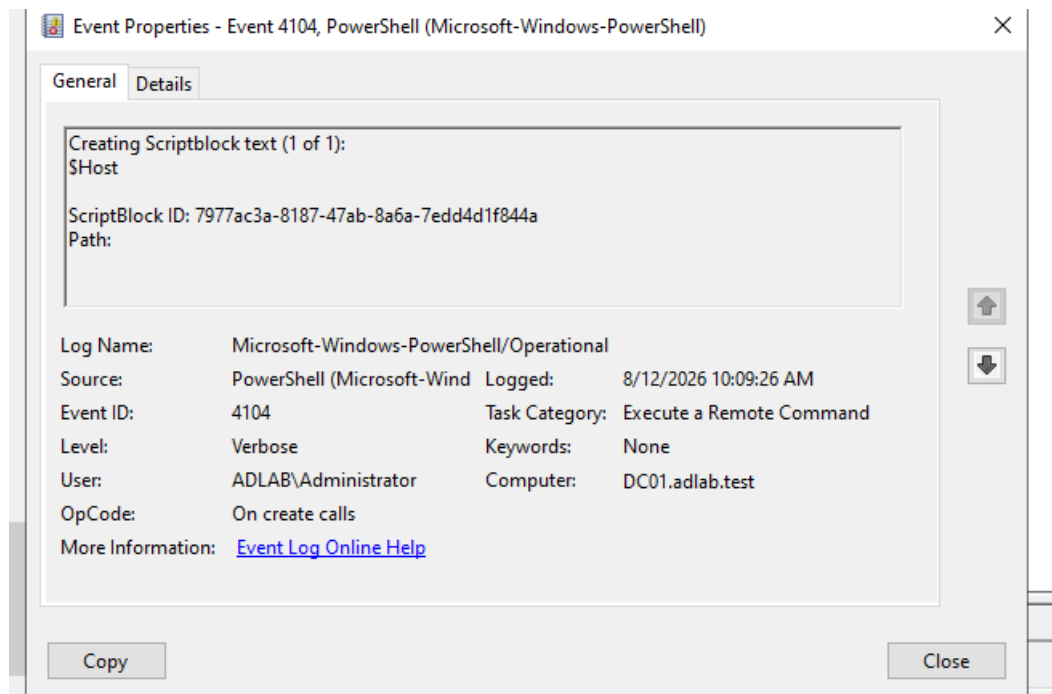


Figure 12: PowerShell 4104 Event

Figure 13 — Event 4104 validating that PowerShell script-block content is available for detection and hunting.

Figure 14 — Wazuh PowerShell hunting view. Analysts should inspect scriptBlockText, endpoint, user, timestamp, and rule metadata when distinguishing administration from discovery behavior.

Figure 15 — Wazuh evidence associated with custom PowerShell rule 110120. This validates the current suspicious-script-block rule, but it should not be treated as proof that the current XML exactly matches the Sigma AD-recon logic.

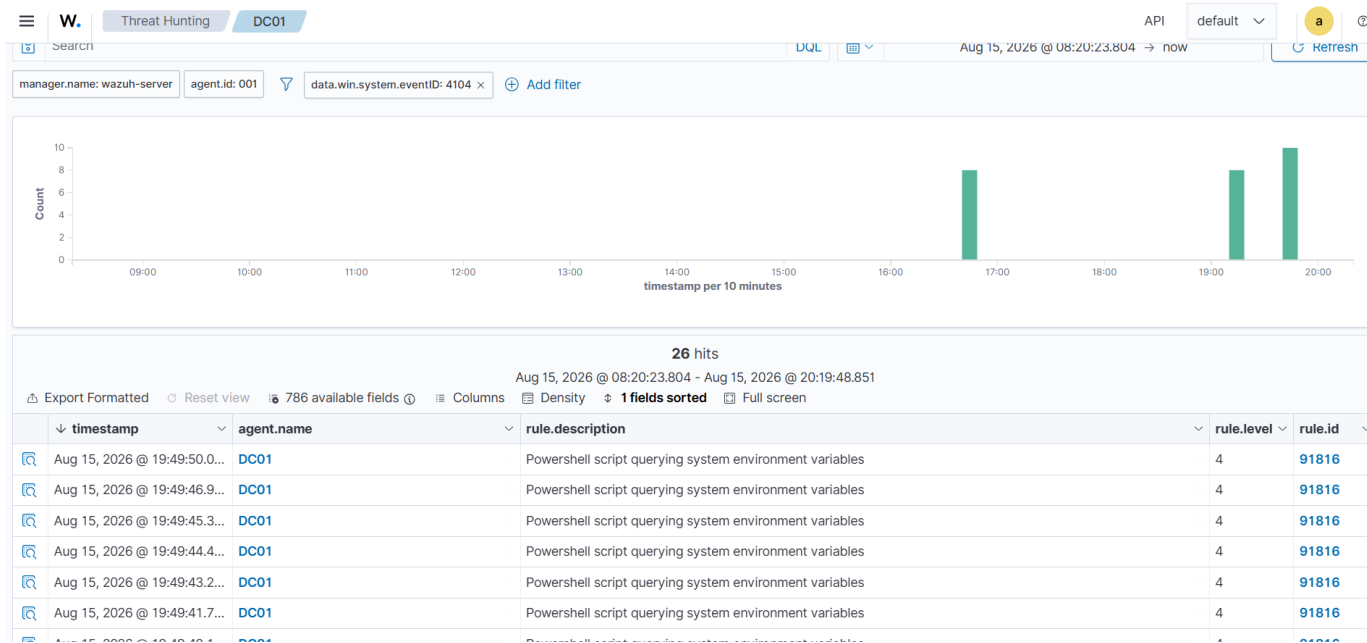


Figure 13: PowerShell Hunting

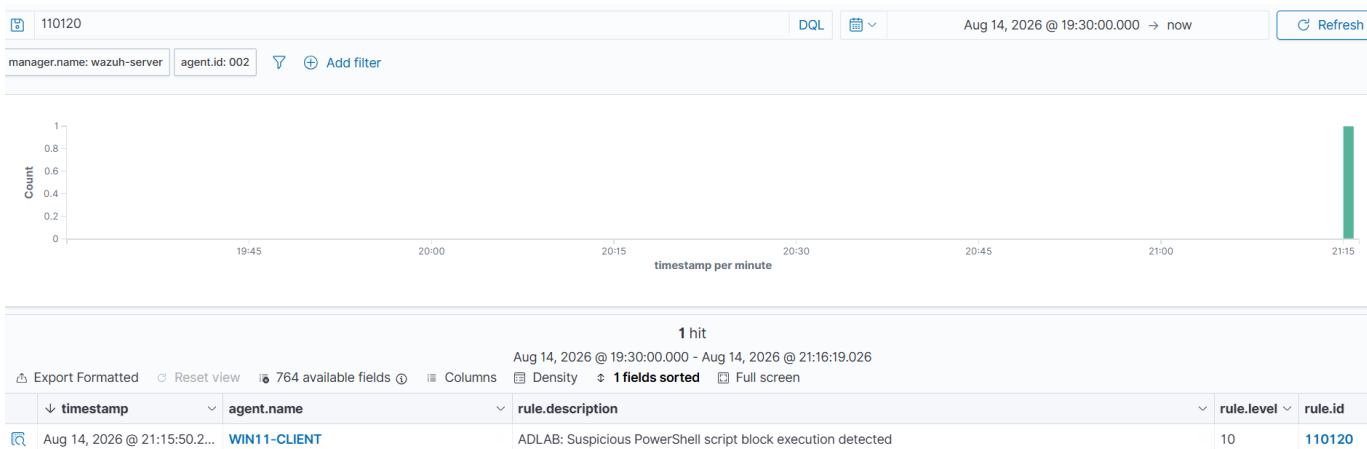


Figure 14: Suspicious PowerShell Alert

4.5 Privileged Group Escalation

Objective

Validate monitoring of a temporary addition to a highly privileged Active Directory group.

Controlled change:

helpdesk.test

↓

Domain Admins

Representative command:

```
Add-ADGroupMember -Identity "Domain Admins" -Members "helpdesk.test"
```

Cleanup:

```
Remove-ADGroupMember -Identity "Domain Admins" -Members "helpdesk.test" -Confirm:$false
```

Telemetry

Primary event:

4728 – Member added to a security-enabled global group

Detection

The validated alert was a Wazuh built-in rule:

Rule ID: 60159

Description: Domain Admins Group Changed

Level: 12

A separate custom Wazuh privilege-change rule is **not** claimed as validated.

Sigma Detection

The repository contains:

detections/sigma/privileged-group-change.yml

Condensed logic:

```
selection_event:
```

```
  EventID:
```

- 4728
- 4732
- 4756

```
selection_group:
```

```
  TargetUserName|contains:
```

- 'Domain Admins'
- 'Enterprise Admins'
- 'Administrators'

MITRE mapping:

T1098 – Account Manipulation

Sigma severity:

high

```

TimeCreated : 8/15/2026 9:56:22 AM
Id          : 4728
Message     : A member was added to a security-enabled global group.

Subject:
  Security ID:      S-1-5-21-36052701-3025511810-1330942903-500
  Account Name:    Administrator
  Account Domain:  ADLAB
  Logon ID:        0x96510

Member:
  Security ID:      S-1-5-21-36052701-3025511810-1330942903-1109
  Account Name:    CN=Helpdesk Test,OU=Lab Users,DC=adlab,DC=test

Group:
  Security ID:      S-1-5-21-36052701-3025511810-1330942903-512
  Group Name:      Domain Admins
  Group Domain:    ADLAB

Additional Information:
  Privileges:      -

```

Figure 15: Privileged Group 4728

Evidence

Figure 16 — Windows Event 4728 generated by the controlled addition of `helpdesk.test` to Domain Admins. Key investigation fields include member identity, target group, subject account, host, and timestamp.

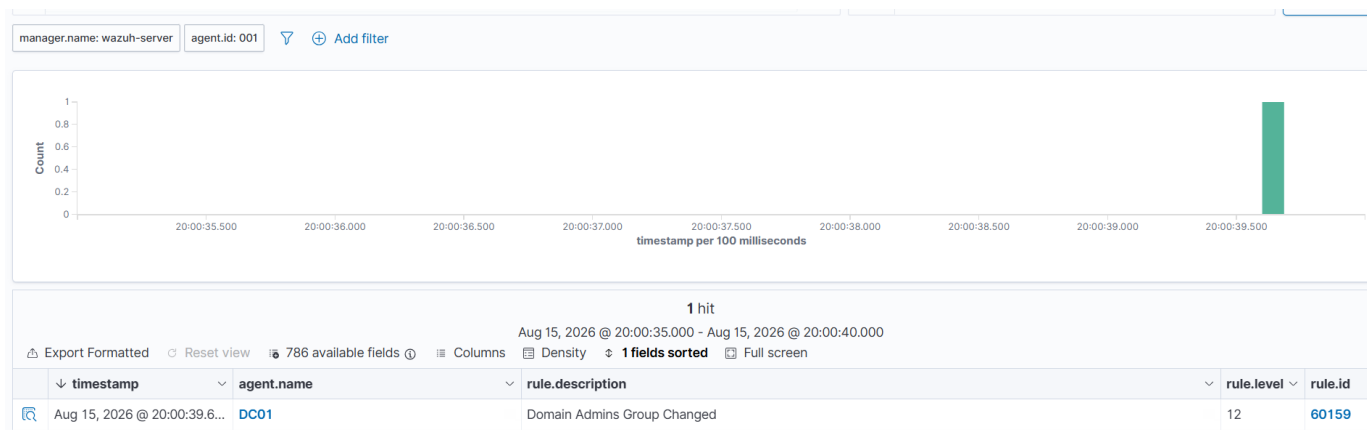


Figure 16: Wazuh Domain Admins Alert

Figure 17 — Wazuh rule 60159 at Level 12, demonstrating high-severity detection of the Domain Admins membership change.

Validation Result

Detected
 Validated Wazuh Rule: 60159
 Level: 12
 Sigma: privileged-group-change.yml
 MITRE: T1098

The temporary privileged membership was removed after evidence collection.

4.6 Deception Mechanism — canary.admin

Objective

Create a high-signal identity with no legitimate authentication workflow so that any attempted use can be prioritized for investigation.

Canary state:

Account: canary.admin

Enabled: False

Privilege: Non-administrative

Expected legitimate authentication: None

Telemetry

Controlled attempt:

Event ID: 4776

Target: canary.admin

Source Workstation: WIN11-CLIENT

Detection Logic

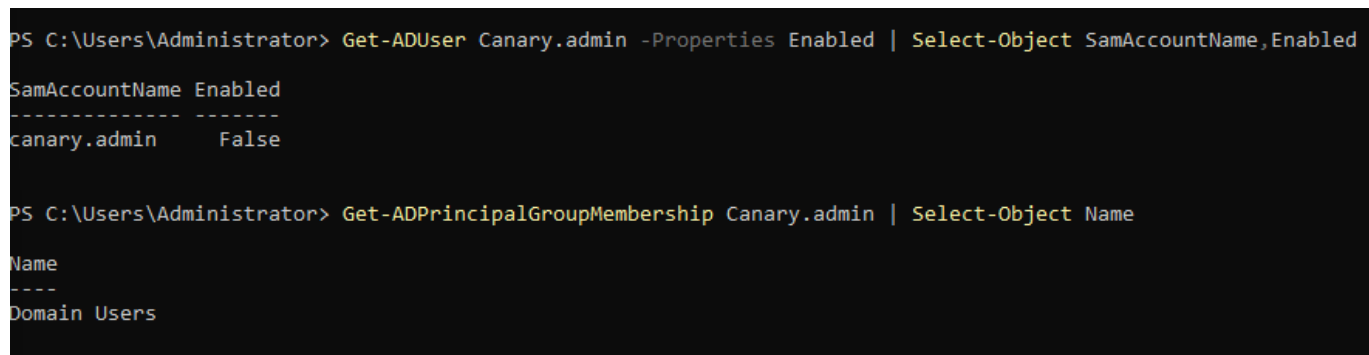
Condensed Wazuh XML:

```
<rule id="110150" level="12">
  <field name="win.system.eventID">^4776$</field>
  <field name="win.eventdata.targetUserName"
    type="pcre2">(?!i)^canary\.admin$</field>
</rule>
```

MITRE context:

T1078 – Valid Accounts

Evidence



```
PS C:\Users\Administrator> Get-ADUser Canary.admin -Properties Enabled | Select-Object SamAccountName,Enabled
SamAccountName Enabled
-----
canary.admin      False

PS C:\Users\Administrator> Get-ADPrincipalGroupMembership Canary.admin | Select-Object Name
Name
----
Domain Users
```

Figure 17: Canary Account Disabled

Figure 18 — Canary-account state confirming that the account is disabled and not intended for normal administration.

Figure 19 — Event 4776 generated by the controlled authentication attempt. The key fields are targetUserName = canary.admin and the originating workstation.

Figure 20 — Wazuh custom rule 110150 at Level 12, proving that the canary authentication event reached the detection layer.

```

TimeCreated : 8/15/2026 9:34:34 AM
Id          : 4776
Message     : The computer attempted to validate the credentials for an account.

Authentication Package:  MICROSOFT_AUTHENTICATION_PACKAGE_V1_0
Logon Account:   canary.admin
Source Workstation: WIN11-CLIENT
Error Code:      0xC000006A

```

Figure 18: Canary 4776

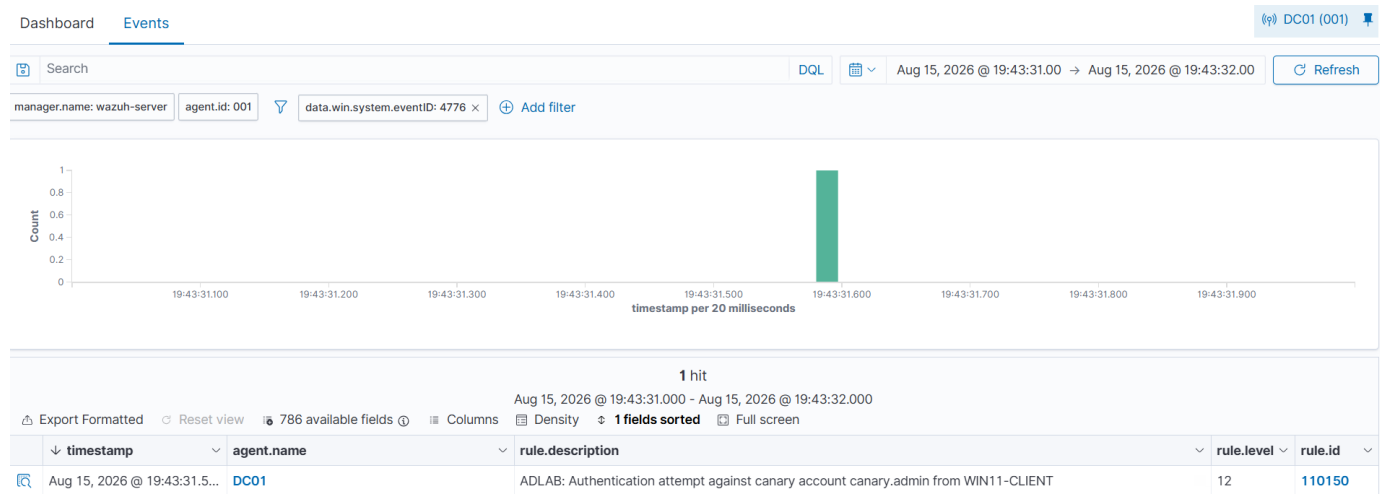


Figure 19: Canary Custom Rule

Validation Result

Detected
 Custom Rule: 110150
 Level: 12
 MITRE context: T1078

The same rule was successfully retested after hardening and while benign background activity was present.

5. Supporting Validation — Remote Administrative Logon

A controlled PowerShell Remoting / WinRM session was generated from:

WIN11-CLIENT
 10.10.10.20

to:

DC01
 10.10.10.10

Observed telemetry:

Event ID: 4624
 Logon Type: 3
 Account: Administrator
 Source IP: 10.10.10.20
 Authentication: Kerberos


```
TimeCreated : 8/15/2026 10:02:05 AM
Id          : 4624
Message     : An account was successfully logged on.

Subject:
  Security ID:      S-1-0-0
  Account Name:     -
  Account Domain:   -
  Logon ID:         0x0

Logon Information:
  Logon Type:       3
  Restricted Admin Mode: -
  Virtual Account:  No
  Elevated Token:   Yes

Impersonation Level:      Impersonation

New Logon:
  Security ID:      S-1-5-21-36052701-3025511810-1330942903-500
  Account Name:     Administrator
  Account Domain:   ADLAB.TEST
  Logon ID:         0x276063
  Linked Logon ID:  0x0
  Network Account Name: -
  Network Account Domain: -
  Logon GUID:       {e4e3a2e2-7ae8-10c7-8a48-61f1e55b33cd}

Process Information:
  Process ID:       0x0
  Process Name:     -

Network Information:
  Workstation Name: -
  Source Network Address: 10.10.10.20
  Source Port:      49927
```

Figure 20: Remote Logon 4624

Figure 21 — Local Windows Security evidence for the remote administrative session. The critical fields are Event 4624, Logon Type 3, administrative identity, and source address 10.10.10.20.

Result

Raw telemetry: Confirmed
Dedicated Wazuh alert: Not confirmed
Status: Logged but not alerted
This is a detection gap, not a successful alert.

6. Telemetry Gaps & Limitations

6.1 Remote Type-3 Logon — Logged but Not Alerted

Observed

4624
Logon Type 3
Administrator
10.10.10.20 → DC01
Kerberos

Gap

An exact dedicated Wazuh alert for the controlled remote administrative event was not confirmed.

Engineering remediation

1. Verify that the Security event channel is explicitly and consistently collected by the Wazuh endpoint configuration.
 2. Build a dedicated detection for:
 - Event 4624
 - Logon Type 3
 - Privileged identity
 - Unusual source workstation or IP
 3. Baseline approved administrative source hosts.
 4. Test the rule with both legitimate administrative traffic and controlled abnormal sources.
 5. Add the scenario to regression testing.
-

6.2 LDAP Event 1644 — Local Event, No Confirmed Wazuh Ingestion

Temporary NTDS Field Engineering diagnostics successfully generated Event 1644 locally.

Figure 22 — Local Directory Service Event 1644. The screenshot demonstrates endpoint-side telemetry generation but does not prove Wazuh ingestion.

Gap

DC01 local telemetry: Confirmed
Wazuh ingestion: Not confirmed

Engineering remediation

- Explicitly configure and validate Directory Service event collection in the Wazuh agent configuration.
 - Generate a controlled LDAP query.
 - Verify the event reaches the manager.
 - Confirm decoding and searchable fields.
 - Only after ingestion is validated should an alert or hunt be marked operational.
-

```
TimeCreated : 8/15/2026 9:44:35 AM
Id          : 1644
Message     : Internal event: A client issued a search operation with the following options.

              Client:
              [::1]:49859
              Starting node:
              CN=Schema,CN=Configuration,DC=adlab,DC=test
              Filter:
              ( & (objectClass=attributeSchema) (LDAPDisplayName=msds-allowedtoactonbehalffotheridentity) )
              Search scope:
              subtree
              Attribute selection:
              attributeSyntax,oMSyntax,oMObjectClass
              Server controls:

              Visited entries:
              1
              Returned entries:
              1
              Used indexes:
              idx_LDAPDisplayName:1:N;
              Pages referenced:
              39
              Pages read from disk:
              1
              Pages preread from disk:
              0
              Clean pages modified:
              0
              Dirty pages modified:
              0
              Search time (ms):
              15
              Attributes Preventing Optimization:
              none
              User:
              ADLAB\DC01$
```

Figure 21: LDAP 1644

6.3 Event 4662 Requires Object-Level SACL Configuration

Event 4662 is not produced simply by enabling Directory Service Access auditing.

The project correctly documents the required combination:

Advanced Audit Policy

+

Object-level SACL

Test object:

OU=Test-Lab,DC=adlab,DC=test

```
PS C:\Users\Administrator> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

PS C:\Users\Administrator> auditpol /get /subcategory:"Directory Service Access"
System audit policy
Category/Subcategory          Setting
DS Access
  Directory Service Access    Success and Failure
PS C:\Users\Administrator> auditpol /get /subcategory:"Directory Service Changes"
System audit policy
Category/Subcategory          Setting
DS Access
  Directory Service Changes    Success
PS C:\Users\Administrator> _
```

Figure 22: Directory Service Audit Policy

Figure 23 — Directory Service Access audit-policy state.

```
PS C:\Users\Administrator> Get-ADUser alice.user -Properties * |
>> Select-Object SamAccountName,Enabled,DistinguishedName

SamAccountName Enabled DistinguishedName
-----
alice.user      True CN=Alice User,OU=Lab Users,DC=adlab,DC=test

PS C:\Users\Administrator>
```

Figure 23: Event 4662

Figure 24 — Local Event 4662 after the object-level auditing requirement was satisfied.

Figure 25 — Wazuh-side evidence that Event 4662 was searchable after targeted auditing was configured.

Operational risk

If the SACL is removed or applied to the wrong object, audit policy can remain enabled while meaningful object-access telemetry silently disappears.

Engineering remediation

- Document the exact monitored OU/object scope.
- Periodically validate the SACL.
- Add a harmless recurring 4662 validation action.
- Treat absence of expected 4662 telemetry as a monitoring-health signal.

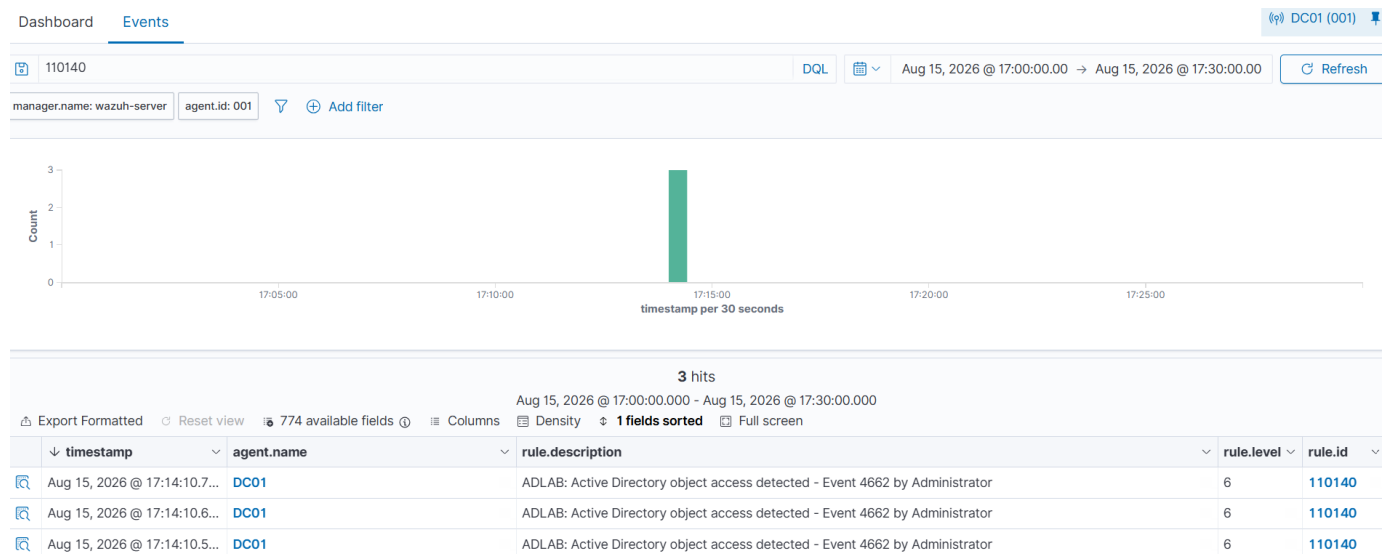


Figure 24: Wazuh 4662

6.4 PowerShell Detection Content Drift

The repository currently contains:

Wazuh 110120:

Suspicious execution/download/encoding terms

and:

Sigma powershell-ad-recon.yml:

Get-ADDomain

Get-ADUser

Get-ADGroup

Get-ADGroupMember

The reports currently associate 110120 with the AD-recon scenario, but the live XML does not implement the same detection logic.

Engineering remediation

Create two explicit detections:

110120 – Suspicious PowerShell execution

110121 – PowerShell Active Directory reconnaissance

Then map:

110120 → T1059.001

110121 → T1087.002 / T1069.002

This removes ambiguity and makes Wazuh/Sigma parity testable.

6.5 Windows Security Collection Is Not Explicit in Repository agent.conf

The current agent.conf explicitly lists:

PowerShell Operational

Sysmon Operational

The project evidence clearly contains Security events such as 4624, 4728, 4769, and 4776, meaning Security telemetry was operational during the lab.

However, the repository does not currently make that collection path explicit.

Engineering remediation

Either:

- add the intended Security channel configuration to the repository; or
- document that collection relies on Wazuh's existing/default Windows Security configuration.

This is primarily a reproducibility and documentation issue.

6.6 Benign-Noise Automation Source Is Missing

The repository currently contains zero-byte files:

```
automation/benign-noise.ps1
automation/scheduled-task-setup.ps1
```

The reports and screenshots demonstrate that benign-noise testing was performed, but the executable source is not preserved in the current main branch.

Engineering remediation

Restore the actual scripts and document:

- execution interval
- generated activity types
- target hosts
- stop/cleanup procedure
- scheduled-task installation/removal
- expected event distribution

This is important for reproducible regression testing.

6.7 Detection Latency Was Not Measured

The project does not claim precise detection latency.

That is correct because consistent timing measurements are not preserved.

Engineering remediation

For future tests, capture:

```
T0 = Activity start
T1 = Windows event timestamp
T2 = Wazuh ingestion timestamp
T3 = Alert timestamp
```

Then calculate:

```
Endpoint generation delay = T1 - T0
Ingestion delay           = T2 - T1
Alerting delay            = T3 - T2
End-to-end detection time = T3 - T0
```

7. Detection Validation Summary

Validation ID	Scenario	Result
ADLAB-001	Failed authentication / password spray	Detected
ADLAB-002	Kerberoasting	Detected

Validation ID	Scenario	Result
ADLAB-003	PowerShell scenario	Telemetry and project detection evidence present; implementation alignment requires cleanup
ADLAB-004	Privileged group change	Detected
ADLAB-005	Canary authentication	Detected
ADLAB-006	Remote administrative logon	Logged but not alerted
ADLAB-007	AD object access	Telemetry validated
ADLAB-008	LDAP diagnostic query	Local-only / ingestion gap

8. False-Positive Validation

The project intentionally introduced benign background traffic so the custom detections were not evaluated only in a silent laboratory.

Documented benign activity included:

- DNS lookups
- ICMP checks
- SYSVOL access
- NETLOGON access
- routine PowerShell activity

Two detections were explicitly retested:

Detection	Controlled Activity	Observed Benign False Positive
110130 — Network reconnaissance	Detected	None observed
110150 — Canary authentication	Detected	None observed

Figure 26 — Background event distribution used to provide non-malicious activity during validation.

Figure 27 — Canary rule 110150 remaining effective while benign activity was active.

The correct conclusion is limited to the test window:

No benign false positive was observed during the controlled validation period.

This is not evidence of a zero false-positive rate in a production enterprise.

9. Hardening & Post-Hardening Retest

Hardening Actions

- Replaced the intentionally weak svc_web password.
- Verified svc_web remained non-administrative.
- Removed helpdesk.test from Domain Admins.
- Kept canary.admin disabled.
- Disabled temporary LDAP diagnostic verbosity.
- Removed the temporary LDAP search threshold.
- Preserved Sysmon and Wazuh monitoring.

Figure 28 — Post-test identity-state evidence validating privilege cleanup and service-account hardening.

273	WIN11-CLIENT	unknown	unknown	
260	WIN11-CLIENT	Microsoft-Windows-Sysmon/Operational		11
237	DC01	unknown	unknown	
180	DC01	Microsoft-Windows-Sysmon/Operational		11
131	DC01	Microsoft-Windows-Sysmon/Operational		3
49	DC01	Security	4624	
47	DC01	Microsoft-Windows-Sysmon/Operational		22
46	DC01	Security	4672	
42	WIN11-CLIENT	Microsoft-Windows-Sysmon/Operational		1
35	WIN11-CLIENT	Microsoft-Windows-Sysmon/Operational		5
30	DC01	System	7036	
27	WIN11-CLIENT	Microsoft-Windows-Sysmon/Operational		3
21	DC01	Security	5379	
14	WIN11-CLIENT	Security	4798	
11	DC01	Microsoft-Windows-Sysmon/Operational		5
10	WIN11-CLIENT	System	10010	
10	DC01	Security	4688	
10	DC01	Microsoft-Windows-Sysmon/Operational		1
9	WIN11-CLIENT	Security	5379	
7	WIN11-CLIENT	Security	4672	
7	WIN11-CLIENT	Security	4624	
7	WIN11-CLIENT	Microsoft-Windows-PowerShell/Operational		4104
6	DC01	Security	4769	
5	WIN11-CLIENT	Microsoft-Windows-Sysmon/Operational		22
2	WIN11-CLIENT	System	566	
2	WIN11-CLIENT	Security	4648	
2	WIN11-CLIENT	Security	4634	
2	WIN11-CLIENT	Application	15	
2	DC01	Security	4768	
1	WIN11-CLIENT	Application	0	

Figure 25: Benign Baseline

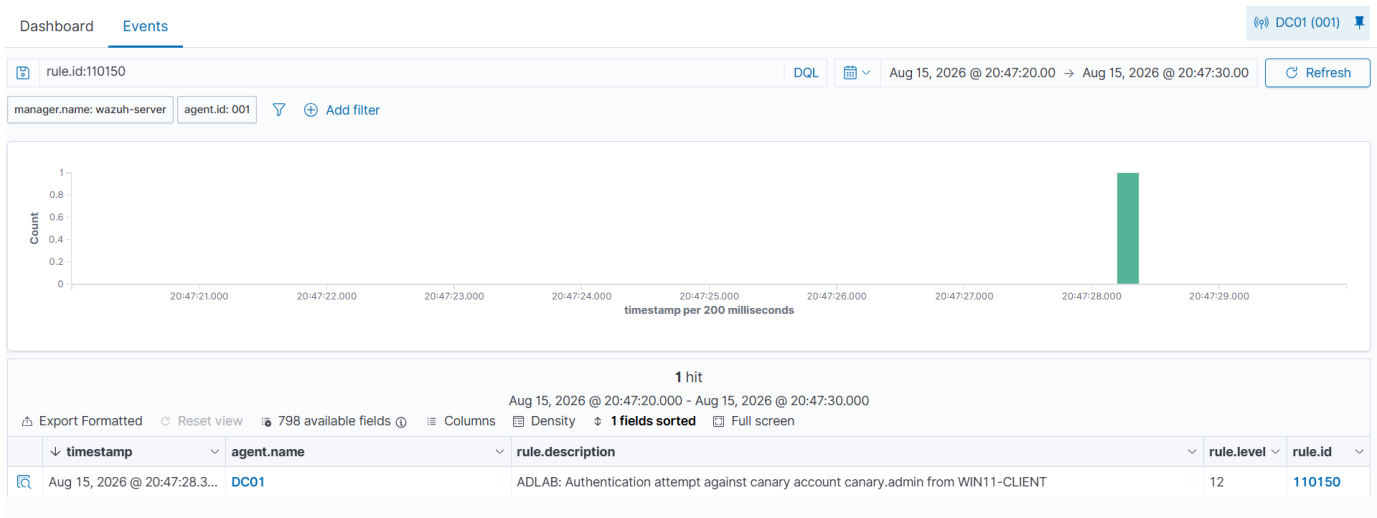


Figure 26: Canary Under Noise


```

PS C:\Users\Administrator> Get-ADPrincipalGroupMembership svc_web | Select-Object Name

Name
----
Domain Users

PS C:\Users\Administrator> Get-ADGroupMember "Domain Admins" |
>> Select-Object Name,SamAccountName

Name          SamAccountName
-----
Administrator Administrator

PS C:\Users\Administrator> Get-ADUser canary.admin -Properties Enabled |
>> Select-Object SamAccountName,Enabled

SamAccountName Enabled
-----
canary.admin    False

```

Figure 27: Hardening State

Retest

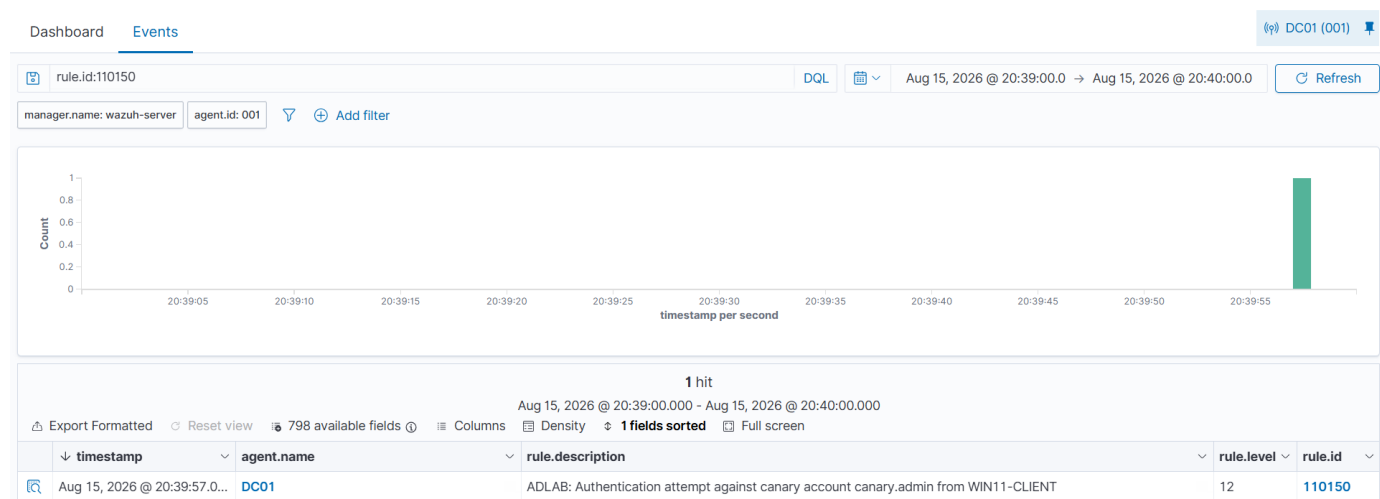


Figure 28: Canary Post-Hardening Retest

Figure 29 — Custom rule 110150 firing after hardening, demonstrating that remediation did not remove the detection path.

Engineering Result

Remediation completed
Monitoring retained
Canary detection revalidated

10. Threat-Hunting Workflows

The repository documents reusable Wazuh queries in:

dashboards/wazuh-saved-searches.md

Representative hunts include:

Authentication

data.win.system.eventID:(4624 OR 4625 OR 4771 OR 4776)

Kerberos

data.win.system.eventID:4769

PowerShell

data.win.system.eventID:4104

Privileged Group Changes

data.win.system.eventID:(4728 OR 4732 OR 4756)

Active Directory Object Activity

data.win.system.eventID:(4662 OR 5136)

Canary

rule.id:110150

Network Reconnaissance

rule.id:110130

These hunts convert one-time testing into repeatable analyst workflows.

11. Engineering Recommendations

Priority 1 — Close Visibility Gaps

- Add and validate a dedicated remote privileged-logon detection.
- Explicitly document / configure Windows Security collection.
- Add Directory Service 1644 ingestion and validation.

Priority 2 — Normalize Detection-as-Code

- Separate suspicious PowerShell execution from AD discovery.
- Align Wazuh and Sigma semantic intent.
- Add automated syntax/schema validation.
- Add event fixtures for regression testing.

Priority 3 — Restore Reproducible Automation

- Restore benign-noise.ps1.
- Restore scheduled-task-setup.ps1.
- Add cleanup instructions.
- Add expected baseline-event profiles.

Priority 4 — Add Measurement

Track:

- event-generation time
- ingestion time
- alert time
- analyst-review time
- false-positive observations

This would turn the existing validation matrix into a measurable detection-quality program.

12. Source Artifacts Reviewed

Detection Rules

- `../detections/wazuh/local_rules.xml`
- `../detections/sigma/kerberoasting.yml`
- `../detections/sigma/powershell-ad-recon.yml`
- `../detections/sigma/privileged-group-change.yml`

Telemetry Configuration

- `../configs/sysmon/sysmon-config.xml`
- `../configs/wazuh/agent.conf`
- `../configs/windows-auditing/advanced-audit-policy.md`
- `../configs/windows-auditing/powershell-logging.md`

Automation

- `../automation/create-lab-users.ps1`
- `../automation/benign-noise.ps1`
- `../automation/scheduled-task-setup.ps1`

Hunting and Validation

- `../dashboards/wazuh-saved-searches.md`
 - `../reports/detection-validation-matrix.md`
 - `../reports/noise-baseline.md`
 - `../reports/test-01-reconnaissance.md`
 - `../reports/test-02-password-spray.md`
 - `../reports/test-03-kerberoasting.md`
 - `../reports/test-04-powershell.md`
 - `../reports/test-05-lateral-movement.md`
 - `../reports/test-06-privilege-change.md`
-

13. Final Engineering Assessment

The AD-LAB demonstrates a credible junior-to-intermediate detection-engineering workflow because it connects attack simulation to actual endpoint telemetry, SIEM correlation, custom rule development, analyst hunting, benign-noise validation, remediation, and post-hardening retesting.

The strongest engineering elements are:

- correlation-based Nmap detection
- correlation-based password-spray detection

- Kerberoasting context using 4769 and RC4 0x17
- high-signal canary authentication
- high-severity privileged-group monitoring
- targeted AD object auditing with 4662
- explicit documentation of failed or incomplete monitoring paths
- false-positive validation against benign activity
- post-hardening detection retesting

The principal remaining work is not adding more attack techniques. It is improving **reproducibility and detection quality** by closing ingestion gaps, aligning detection-as-code implementations, restoring automation source, and adding measurable detection latency and regression testing.

Security Disclaimer

This report documents authorized testing performed inside the isolated AD-LAB environment.

No production infrastructure, public systems, real organizational credentials, or destructive payloads were used.